

CDS: An Anti-Aging Calibratable Digital Sensor for Detecting Multiple Types of Fault Injection Attacks

Zhiyuan Chen, Kun Yang* and Kui Ren

The State Key Laboratory of Blockchain and Data Security, Zhejiang University
School of Cyber Science and Technology, Zhejiang University
ZJU-Hangzhou Global Scientific and Technological Innovation Center

ABSTRACT

Fault injection attacks (FIAs) are a class of active physical attacks that inject faults into computing devices to deliberately change their intended behaviors for malicious purposes such as security feature circumvention, privilege escalation, secret data extraction by analyzing the erroneous outputs, etc. Existing sensors that detect fault injection attacks are either susceptible to aging or suffer from complicated and costly calibration process. Worse still, most sensors will fail in the presence of dynamic voltage and frequency scaling (DVFS) because they are built upon fixed delay chains and cannot adapt to operating voltage and frequency variations. In this paper, we overcome these limitations by presenting CDS, a delay chain based digital sensor that exploits timing variations of both detector and protected object for detecting multiple types of fault injection attacks. CDS utilizes a calibration module to solve the accuracy degradation caused by aging phenomena and a dynamic adjustment module to acclimatize itself to the need of dynamically adjusting voltage according to operating frequency in the presence of DVFS. To demonstrate its capability, we use CDS to protect the hardware accelerator of PRESENT cryptographic algorithm against voltage and laser glitching attacks. Simulation results based on HSPICE show that (i) CDS can detect 100% of voltage and temperature coordinated glitching attacks with 4.1% early warning; (ii) CDS can detect 100% of laser glitching attacks with 9.1% early warning; (iii) CDS maintains outstanding aging resistance with only 1.1% false alarm rate increase after 7 years of use.

1 INTRODUCTION

Fault injection attacks (FIAs) can bypass security mechanisms, alter register contents, and disrupt the normal operations of integrated circuits (ICs) by manipulating the voltage and clock inputs, injecting electromagnetic glitches, etc. to gain higher security privileges [17] or execute malicious codes [16, 19]. With the increased use of field programmable gate array (FPGA) cloud services, attackers begin attempting to conduct remote FIAs. The authors in [9] successfully hammered AES hardware accelerator deployed on FPGA in the cloud by exploiting power consuming circuitry that is connected to

the FPGA's power distribution network (PDN). Subsequently, the authors in [11] presented a successful attack against a deep neural network (DNN) running on a cloud FPGA by utilizing power failure injection. FIAs have been proven to be feasible and pose a threat to the secure operations of computing devices [3, 8]. Therefore, there is an urgent need to develop countermeasures for combating FIAs.

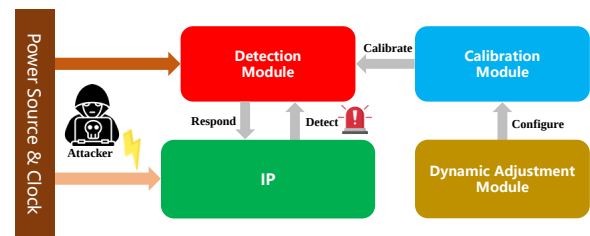


Figure 1: Our proposed counter strategy

There have been many studies proposing strategies to resist FIAs, which can be divided into two categories: (i) redundant computing based strategies and (ii) threshold detection based strategies. The strategies of adding redundancy are taking advantage of the strict requirements of FIAs towards injection timing and the difficulty of triggering multiple identical faults in different logic units or code segments. Redundant computing based countermeasures will either affect performance of computing devices or incur additional hardware overhead. Typical examples of threshold detection based countermeasures include digital [1, 5–7, 12, 14, 21, 22] or analog [4, 13, 15] sensors designed for anomaly detection. Digital sensors are usually designed for detecting abnormal behaviors with regards to circuitry timing given the fact that FIAs introduce faults to program execution mainly by creating timing violation in ICs [12, 20]. Analog sensors usually detect FIAs by directly checking the voltages of power rails, or capturing the voltage pulses induced by varying electromagnetic fields, and compare with the predefined thresholds. Analog sensors have poor portability and tend to consume more overhead in terms of area and power. Digital sensors are therefore becoming the primary choice in the fight against the FIAs.

However, there are many problems with existing digital sensors. Firstly, existing delay chain based digital sensors only use a single D flip-flop (DFF) to generate an alarm signal, which is susceptible to metastability due to setup time constraint. Secondly, digital sensors are prone to aging like analog sensors, which degrades detection accuracy. Lastly, most digital sensors are not designed to be adaptable to the presence of DVFS. To address these issues, we propose a FIA counter-strategy namely CDS as shown in Figure 1. CDS is designed to be an anti-aging digital sensor that can be automatically calibrated. The CDS detection module is designed based on a delay chain and utilizes two complementary DFFs to generate an alarm

*Kun Yang is the Corresponding Author (Email: kuny@zju.edu.cn)

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06

<https://doi.org/10.1145/3649329.3657322>

signal. CDS also includes a calibration module that mitigates aging effects and a dynamic adjustment module that acclimatizes CDS to varying operating voltages and frequencies in the presence of DVFS. The contributions of this paper are summarized as follows:

- To address the threat of fault injection faced by ICs, we propose CDS, an aging-resistant calibratable digital sensor to detect multiple types of FIAs (e.g., voltage glitching, laser glitching, etc.), and demonstrate its detection capability through simulation (100% detection coverage and 4.1% early warning).
- We study in detail the impact of aging phenomena on digital sensors and demonstrate the aging resistance of CDS detection module alone through simulation (only 1.93% false alarm rate after 7 years of use).
- To address the aging problem of digital sensors and to adapt to varying voltages and frequencies in the presence of DVFS, we design calibration module and dynamic adjustment module, which enable automated aging calibration as well as runtime configuration of the delay time of the delay chain. With calibration module and dynamic adjustment module introduced, the false alarm rate after 7 years of use is decreased from 1.93% to 1.1% (i.e., 43% improvement).

2 RELATED WORK

In recent years, there have been many studies proposing different kinds of sensors to detect FIAs, which can be mainly categorized into digital and analog sensors.

Digital Sensors: Digital sensors consist of two main types, delay based digital sensors [1, 6, 7, 12, 14, 21, 22] and impulse based digital sensors [5]. Delay based digital sensors need to design a delay chain. They usually use clock signal as the input to the delay chain and a DFF to capture timing variations caused by FIAs. Impulse based digital sensors are usually composed of several DFFs. Because studies have shown that DFFs are the most vulnerable units to attack and error. They are more like a trap circuit and have no direct timing relationship with the protected circuit. Therefore their detection accuracy is relatively low. Digital sensors usually face the aging problem. Even if the aging calibration function is considered in [2], the calibration cost is relatively high. Besides, most of the existing digital sensors does not take into account the DVFS requirements of the IC. So when the voltage and clock frequency are varying, the accuracy of these sensors will be reduced.

Analog Sensors: Analog sensors [4, 13, 15] usually directly detect certain information about the IC, such as temperature, voltage, etc, and then compare it to a predefined threshold. An analog sensor can only detect one physical quantity and therefore cannot detect multiple types of fault injection attacks at the same time. Analog sensors have to be altered to suit the different design environment and tend to take up more resources. Besides, analog sensors rely on costly calibration schemes, e.g., chip-by-chip laser trimming.

3 PRELIMINARIES

In this section, we will give a short overview of different types of FIAs and their working principles. The basic principles regarding IC aging will also be described.

3.1 Fault Injection Attacks

Typically, fault injection attacks can be categorized as non-intrusive fault attacks, semi-intrusive fault attacks and intrusive fault attacks.

Non-intrusive Fault Attacks: This type of attack does not require modification of the targeted device to induce a fault, making it highly probable that the victim will not recognize the attack. Additionally, these types of attacks do not require sophisticated equipment. Common methods include the use of electromagnetic pulses, positive or negative spikes in the power line, and adding additional clock edges.

Semi-intrusive Fault Attacks: To conduct a semi-intrusive fault injection, it is necessary to breach the attacked device. In optical fault attacks, this modification entails unpacking the chip to access the chip die without disrupting the passivation layer. Both laser and flash light can trigger a fault. Laser can precisely target specific memory or register bits, while flash lights cause global faults across larger regions of the chip.

Intrusive Fault Attacks: Penetration of the attacked device is required for this type of attack. After de-packaging, the passivation layer must be removed to access the metal layer using microprobes. This method enables targeting individual bus lines on the chip and modifying their values. Intrusive fault attacks necessitate the most advanced equipment, with the benefit of minimal limitations for fault injections.

3.2 Timing Constraints

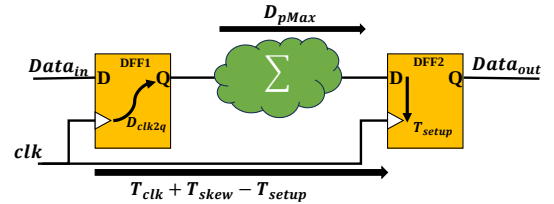


Figure 2: IC internal architecture

In ICs, clock signal is usually used to synchronize the overall operation. Figure 2 shows a brief architecture inside the IC. When the rising edge of the clock signal arrives, data is fed into the combinational logic (labeled Σ) from the upstream DFFs. After a series of logic operations, the results are latched to the downstream DFFs at the next rising edge. IC must satisfy timing constraints to work correctly. The exact writing of the timing constraint formula can be expressed as:

$$T_{clk} > D_{clk2q} + D_{pMax} + T_{setup} - T_{skew} \quad (1)$$

where T_{clk} and D_{pMax} are the clock period and the maximum data propagation time respectively, D_{clk2q} denotes the delay that elapses between the rising edge of the clock and the actual update of the DFF outputs, T_{skew} is the possible skew or small phase difference between the clock signals at the clock inputs of the two different DFFs, and T_{setup} indicates the setup time.

FIAs work by violating the timing constraints. One way to directly violate them is to overclock the circuit or inject a very short clock period, causing the circuit's clock period to decrease as formula (2). This will lead to DFF setup time violation or premature data latches. Another way to violate the timing constraints is to increase the processing time of the combinational logic to ensure

correct operation as formula (3). Injection voltage glitchings and electromagnetic pulses can all lead to increased processing times of combinational logic.

$$T_{clk_{dec}} < D_{clk2q} + D_{pMax} + T_{setup} - T_{skew} \quad (2)$$

$$T_{clk} < D_{clk2q} + D_{pMax_{inc}} + T_{setup} - T_{skew} \quad (3)$$

where $T_{clk_{dec}}$ denotes the shortened clock period, and $D_{pMax_{inc}}$ denotes the increased maximum propagation time due to the attack.

3.3 Integrated Circuits Aging

The two main factors of aging phenomenon in CMOS technology are negative bias temperature instability (NBTI) and hot carrier injection (HCI). They lead to increase in the delay of signal propagation in ICs.

NBTI Aging: Bias temperature instability (BTI) is a shift in threshold voltage with applied stress. For pMOS transistors, the threshold voltage corresponds to a negative gate bias. NBTI is a more serious concern than positive BTI. Transistors undergo two NBTI phases depending on their operating state. The stress phase happens when the transistor is on ($V_{gs} < V_{th}$). The threshold voltage of the transistor increases during this phase due to the Si-SiO₂ interface generating positive interface traps. The second recovery phase happens when the transistor is off ($V_{gs} > V_{th}$). The threshold voltage drift that occurs during the stress phase partially recovers in the recovery phase.

HCI Aging: HCI happens when carriers are injected into the gate dielectric during transistor switching, creating interface traps, oxide charges, and parasitic currents. HCI is linked to switching activity, leading to circuit degradation with shifting threshold voltage and drain current of stressed transistors. Typically, nMOS transistors face effects from HCI. The induced threshold voltage drift caused by HCI is sensitive to the number of gate input transitions of the transistor.

4 CDS SENSOR

The proposed CDS sensor is presented in this section. We first describe the overall architecture of CDS sensor, and then introduce the specific functionalities and design details of its building blocks.

4.1 Architecture Overview

Figure 3 shows the overall architecture of CDS sensor, which consists of three main functional modules: (i) **Detection Module:** the detection module of CDS sensor is built upon a delay chain and is capable of detecting multiple types of fault injection attacks by capturing the timing variation of the delay chain; (ii) **Calibration Module:** the calibration module of CDS sensor also includes a delay chain and enables automated aging calibration with accuracy of one basic delay unit; (iii) **Dynamic Adjustment Module:** the dynamic adjustment module works together with the calibration module to acclimatize CDS to the need of dynamically adjusting voltage according to operating frequency in the presence of DVFS.

4.2 Detection Module

As shown in the upper half of Figure 3, the detection module of CDS sensor mainly consists of an artificial delay chain, two DFFs,

and an OR gate. The artificial delay chain includes a series of series-connected basic delay units. Each basic delay unit is composed of a buffer and a 2-to-1 multiplexer. The length of the delay chain is configured on the basis of the critical path of the protected IP. The length of the delay chain *delay* is usually set to be slightly larger than the maximum data propagation time through the logic D_{pMax} so as to play an early warning role.

The detection principle of CDS is shown in Figure 4, where clk and D_{clk} indicate the clock signal and the clock signal after the delay chain respectively. When the computing device works normally, the delay of the delay chain is larger than the maximum data propagation time through the logic and smaller than the clock period, in which case CDS will not issue an alarm. FIA will normally increase *delay* and D_{pMAX} . When the clock period is larger than the maximum data propagation time through the logic and smaller than the delay of the delay chain, the protected device will not generate an erroneous output and CDS will issue an early warning. When the maximum data propagation time through the logic is larger than the clock period and smaller than the delay of the delay chain, the protected device will work incorrectly and CDS will issue an alarm. CDS uses two DFFs to capture timing variation. In some previous work [7, 21], often only one DFF is used for capturing timing variation and the capturing DFF itself will also be affected by timing violation, which will result in detection blind zone of the sensor. CDS solves this problem by making two DFFs work complementarily for capturing timing variation, and finally generates an alarm signal via an OR gate if FIA happens.

4.3 Calibration Module

The calibration module, as shown in the lower left of Figure 3, consists of a frequency division DFF, a delay chain, and a series of calibration DFFs. Each calibration DFF is serving for a corresponding basic delay unit in the detection module. When CDS is not aging, the delay chain length of the calibration module and the delay chain length of the detection module add up to one clock cycle. When the detection accuracy of CDS decreases due to aging, the calibration signal is set to 1. The frequency division DFF will first halve the clock frequency.

The CDS calibration principle is illustrated in Figure 5, where clk^* indicates to the clock signal after frequency division, and $D_{clk_1^*}$ and $D_{clk_2^*}$ respectively refer to the data inputs of the i_{th} calibration DFF in case A and case B. In case A, $delay < T_{clk}$, the i_{th} calibration DFF will select the buffer of the i_{th} basic delay unit. In case B, $delay > T_{clk}$, the i_{th} calibration DFF will abandon the buffer of the i_{th} basic delay unit. When aging occurs, the delay time of the delay chain in the detection module will increase. To compensate for the aging effect, the calibration module will automatically abandon a certain number of basic delay units of the detection module's delay chain so that the delay time of the actual active delay chain in the detection module can maintain its initial value. An NMOS sleep transistor and a PMOS sleep transistor are respectively connected to the power terminal and the ground terminal of every component in the calibration module so that the calibration module can be disconnected from the power supply during hibernation to minimize the aging effect. The calibration signal and its complementary signal are respectively connected to the gate terminals of NMOS

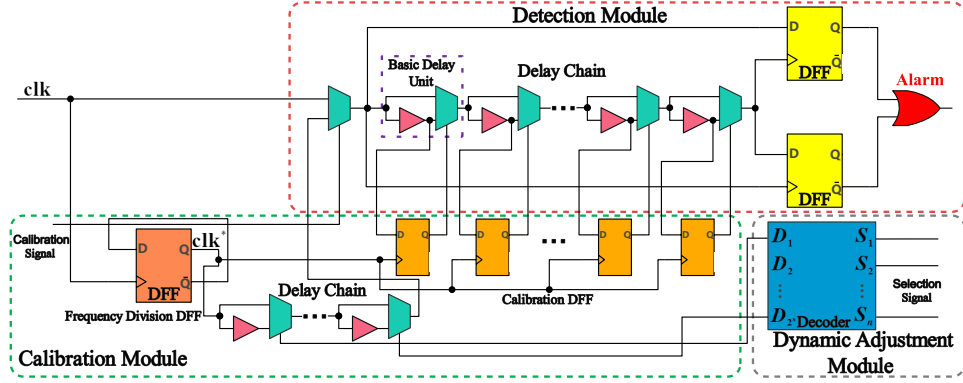


Figure 3: CDS architecture, including detection module, calibration module and dynamic adjustment module

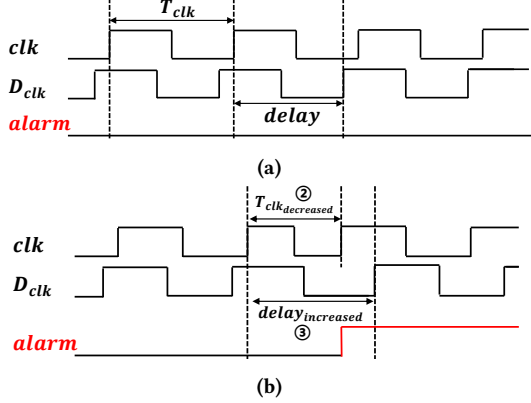


Figure 4: CDS detection principle : (a) normal operation (b) FIAs detection

sleep transistors and PMOS sleep transistors. When the calibration signal is set to 0, the calibration module goes to hibernate.

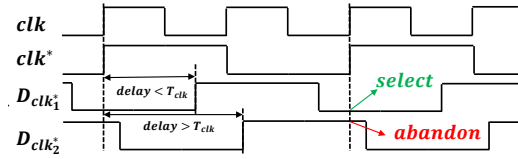


Figure 5: CDS calibration principle

4.4 Dynamic Adjustment Module

As shown in the lower right of Figure 3, the dynamic adjustment module in essence is a decoder, which allows for provisioning a set of selection signals so as to configure the length of the delay chain in the calibration module. The set of output signals of the decoder serve as the set of selection signals for the 2-to-1 multiplexers of the basic delay units in the calibration module, which enables the delay time adjustment of the calibration module's delay chain. After configuring the delay time of the delay chain in the calibration module, the calibration operation as described in Section 4.3 is subsequently performed to dynamically adjust the delay time of the delay chain in the detection module so as to acclimatize CDS to the need of dynamically adjusting voltage according to operating frequency in the presence of DVFS. Although some prior sensors

[7] also allow for adjusting the delay time of delay chain, they do not support aging calibration and will fail in the presence of DVFS since they can no longer accurately determine the delay time of the basic delay unit after aging and thus cannot reconfigure the delay time of the delay chain as needed. The dynamic adjustment module also connects to sleep transistors in order to minimize aging effects.

5 EVALUATION AND SIMULATION RESULTS

In this section, we present the simulation setup and evaluation results regarding detection capability, aging resistance, and calibration effect. Afterwards, we compare CDS with prior work in terms of overhead. Lastly, we perform the attack analysis.

5.1 Simulation Setup

To evaluate the ability of CDS to detect FIA, we choose the PRESENT algorithm's substitution box (S-Box) as the protected object. CDS and the target S-Box are implemented in the transistor level using 45 nm NANGATE technology in HSPICE and simulated for voltage and temperature coordinated glitching attacks and laser glitching attacks [6]. The detailed simulation parameter settings are shown in Table 1. In order to test the effects of aging on the CDS, we use the HSPICE built-in MOSRA Level 3 model to assess the effect of NBTI and HCI aging, up to 7 years of operation in steps of four months.

Table 1: Simulation parameter settings

Parameter	Section 5.2	Section 5.3
Clock Cycle	0.25 ns	5 ns
Temperature*	-10 – 150 °C	-10 – 150 °C
Voltage*	0.8 – 1.2 V	1.0 – 1.4 V
Voltage Glitching Cycle*	0.25 ns	5 ns
Working Environment Point ^Δ	(1 V, 50 °C)	(1.2 V, 55 °C)
PGN Resistance ^Δ	1 – 100 Ω	1 – 100 Ω
PGN Capacitance ^Δ	100 – 1000 fF	100 – 1000 fF
Laser Glitching Cycle ^Δ	0.5 ns	60 ns
PGN Induced Current ^Δ	1 mA	1 mA

* voltage and temperature coordinated glitching attack

^Δ laser glitching attack

5.2 Detection Capability Evaluation

The FIA detection capability of CDS is illustrated in Figure 6. First, we performed two types of FIAs on the target at the same time (i.e., temperature and voltage fault injection). From Figure 6a, the erroneous operation of the S-Box increases with rising temperature and decreasing value of voltage glitching. The S-Box has completely failed to operate properly when the value of voltage glitching is 0.8 V. The simulation results show that the CDS can detect 100% FIAs when the S-Box generates an error output. CDS gives an early warning (33 of 805 attacks) because *delay* of the detection module is slightly longer than D_{pMax} . Figure 6b demonstrates CDS's detection capability of laser fault injection. When $R_{PGN} \geq 40\Omega$, the S-Box works incorrectly, while the CDS can detect 100% of the attacks. When $R_{PGN} = 30\Omega$, the S-Box operating normally and the CDS will issue an early warning signal. It is important to note that CDS does not issue alarm signals when FIAs are not performed. Therefore, according to the simulation results, CDS has great FIA detection accuracy and have a certain degree of early warning function.

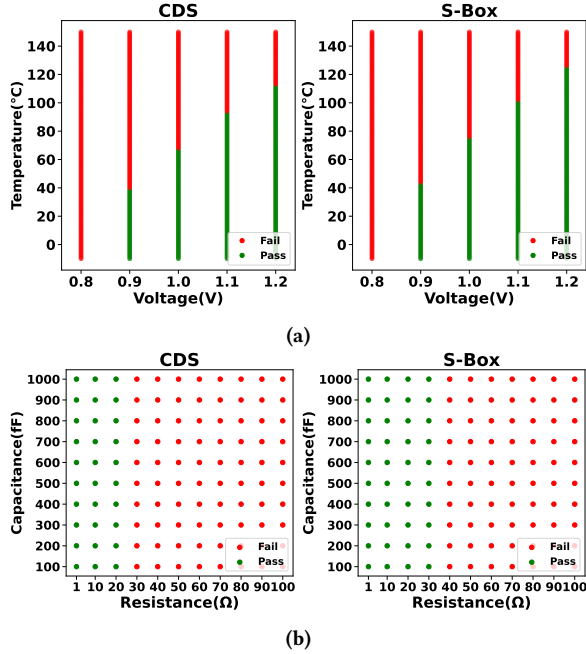


Figure 6: CDS detection capability evaluation: (a) voltage and temperature injection attack detection (b) laser injection attack detection

5.3 Aging Resistance Evaluation

According to [10, 18], the effects of NBTI and HCI on the circuit are mainly related to voltage and temperature. As shown in Figure 7, the delay time of the delay chain increases with time of use. It is about 15 ps after one year of use and about 45 ps after seven years. However, the effect of different operating voltages on the delay time is not as large as expected. Since the detection principle of CDS is by detecting the variations of delay time, the aging phenomena will have some influence on the detection accuracy of CDS. Figure 8 illustrates the degradation in CDS's accuracy for voltage and

temperature coordinated glitching attack detection after 7 years of use. The increase in delay time of the delay chain due to aging is superimposed on the increase in delay time due to FIAs. So CDS generates alarm signals in situations where it otherwise would not. However CDS false alarm rate is only 0.83% after one year of use and 1.93% after seven years of use. As a comparison, we

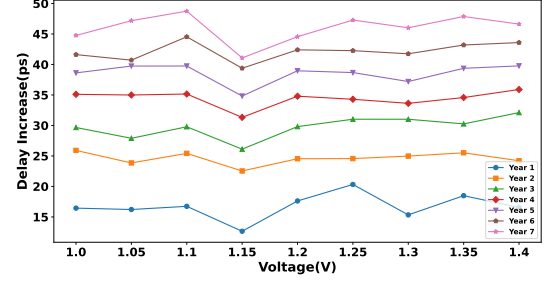


Figure 7: Increased delay of the delay chain

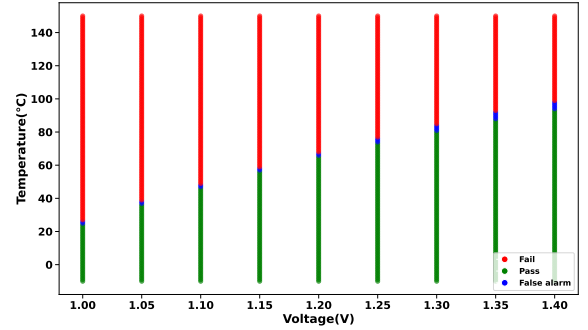


Figure 8: CDS detection capability after 7 years of use

studied the state-of-the-art digital sensor [1] subjected to the aging phenomenon. Since its detection principle is based on detecting changes in environmental parameters, we used the value of the limiting environmental parameter variations as its point of false alarm. The experimental results, as shown in Table 2, show that CDS clearly has better anti-aging properties.

Table 2: Error rate comparison

Time (years)	1	2	3	4	5	6	7
CDS (%)	0.83	1.10	1.17	1.52	1.59	1.79	1.93
[1] (%)	1.17	2.69	4.07	5.66	6.97	7.87	9.67

5.4 Calibration Effect Evaluation

Although our proposed sensors have good aging resistance, we still propose a CDS calibration module to further minimize the effect of aging phenomenon. And the calibration module also serves the dynamic regulation module to meet the DVFS requirement of the IC. The increase in delay due to aging is not significant when the usage time is short, so we choose to calibrate the sensor from the 60th month. The experimental results in Table 3 show that the false alarm rate of the calibrated sensor decreases to a certain extent, and the decrease is more obvious with the increase of the usage time.

Table 3: Error rate comparison between pre-calibrated CDS and post-calibrated CDS

Time (months)	60	64	68	72	76	80	84
Before (%)	1.59	1.66	1.73	1.79	1.79	1.79	1.93
After (%)	1.38	1.31	1.24	1.24	1.17	1.10	1.10

5.5 Overhead Comparison

Table 4 compares CDS with prior work [1, 6, 22] in terms of consumed equivalent gate counts. According to our design, the sum of the delay time of delay chain in the CDS detection module and the delay time of delay chain in the CDS calibration module is always equal to one clock cycle. Therefore, as shown in Table 4, the actual active equivalent gate counts consumed by the CDS detection module as well as the CDS sensor will vary with the clock frequency. When the normal operating frequency of protected computing device is high, the area overhead of CDS sensor is much smaller than prior sensors. The advantage in terms of area overhead will be much more significant if we only count the equivalent gate count consumed by the detection module given most prior sensors do not provide aging calibration or dynamic adjustment functionality.

Table 4: Overhead comparison

Sensor	CDS _{0.25ns}	CDS _{5ns}	[1]	[6]	[22]
Equivalent gate count	95 (47*)	505 (197*)	384	292	184

count*: equivalent gate count consumed by the CDS detection module

5.6 Attack Analysis

Many studies [13, 22] have shown that almost all FIAs make the target device malfunction by causing timing violations. The CDS proposed in this paper is a digital sensor based on timing variations detection and can defend against most of the FIAs. CDS cannot defend against one type of attacks that directly tamper with the OR gate which generates the alarm signal, which will be addressed in our future work. However, since CDS only occupies a very small area, attackers would have to use very sophisticated equipment and a significant amount of time to reverse engineer the IC layout and further determine the specific location of the OR gate.

6 CONCLUSION AND FUTURE WORK

In this work, we proposed CDS, a calibratable and dynamically adjustable digital sensor that detects multiply types of fault injection attacks including laser and voltage glitching. Compared with most existing FIA detectors that only exploit timing variation of detectors themselves, CDS has better aging resistance because of the following two merits: (i) CDS makes decisions based on timing variation of both detector and protected object; (ii) calibration module and dynamic adjustment module of CDS will only work during calibration phase and will hibernate during detection phase by being disconnected from power using sleep transistors. Furthermore, CDS can automatically calibrate and dynamically adjust itself to adapt to varying operating voltages and frequencies in the presence of DVFS. Simulation results based on HSPICE has demonstrated its detection capability, self-calibration and self-adjusting abilities,

and aging resistance. In future work, we plan to tape out a security chip with CDS integrated and test it in a laboratory environment.

ACKNOWLEDGMENTS

This work was supported in part by the National Key Research and Development Program of China (Grant No. 2023YFB3105900), the National Natural Science Foundation of China (Grant No. 62372407) and the Fundamental Research Funds for the Central Universities (Grant No. 226-2023-00030).

REFERENCES

- [1] Anik Md Toufiq Hasan et al. 2020. Detecting Failures and Attacks via Digital Sensors. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40, 7 (2020), 1315–1326.
- [2] Anik Md Toufiq Hasan et al. 2021. Reducing Aging Impacts in Digital Sensors via Run-Time Calibration. *Journal of Electronic Testing* 37, 5-6 (2021), 653–673.
- [3] Breier Jakub et al. 2022. How Practical Are Fault Injection Attacks, Really? *IEEE Access* 10 (2022), 113122–113130.
- [4] Bastos R Possamai et al. 2013. A Bulk Built-In Sensor for Detection of Fault Attacks. In *2013 IEEE International Symposium on Hardware-Oriented Security and Trust (HOST)*. IEEE, 51–54.
- [5] El-Baze David et al. 2016. A Fully-Digital EM Pulse Detector. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 439–444.
- [6] Ebrahimabadi Mohammad et al. 2022. Detecting Laser Fault Injection Attacks via Time-to-Digital Converter Sensors. In *2022 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 97–100.
- [7] Endo Sho et al. 2012. An Efficient Countermeasure Against Fault Sensitivity Analysis Using Configurable Delay Blocks. In *2012 Workshop on Fault Diagnosis and Tolerance in Cryptography*. IEEE, 95–102.
- [8] Gangolli Aakash et al. 2022. A Systematic Review of Fault Injection Attacks on IoT Systems. *Electronics* 11, 13 (2022), 2023.
- [9] Krautter Jonas et al. 2018. FPGAhammer: Remote Voltage Fault Attacks on Shared FPGAs, Suitable for DFA on AES. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2018), 44–68.
- [10] Lahbib Insaf et al. 2015. Hot Carrier Injection Effect on Threshold Voltage of NMOSFETs. In *2015 11th Conference on Ph. D. Research in Microelectronics and Electronics (PRIME)*. IEEE, 164–167.
- [11] Luo Yukui et al. 2021. DeepStrike: Remotely-Guided Fault Injection Attacks on DNN Accelerator in Cloud-FPGA. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 295–300.
- [12] Muttaki Md Rafid et al. 2022. FTC: A Universal Sensor for Fault Injection Attack Detection. In *2022 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 117–120.
- [13] Miura Noriyuki et al. 2016. PLL to the Rescue: A Novel EM Fault Countermeasure. In *Proceedings of the 53rd Annual Design Automation Conference*. 1–6.
- [14] Pundir Nitin et al. 2022. Security Properties Driven Pre-Silicon Laser Fault Injection Assessment. In *2022 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 9–12.
- [15] Sim Jae-Yoon et al. 2003. A 1.8-V 128-Mb Mobile DRAM with Double Boosting Pump, Hybrid Current Sense Amplifier, and Dual-Referenced Adjustment Scheme for Temperature Sensor. *IEEE Journal of Solid-State Circuits* 38, 4 (2003), 631–640.
- [16] Timmers Niek et al. 2016. Controlling PC on ARM Using Fault Injection. In *2016 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC)*. IEEE, 25–35.
- [17] Timmers Niek et al. 2017. Escalating Privileges in Linux Using Voltage Fault Injection. In *2017 Workshop on Fault Diagnosis and Tolerance in Cryptography (FDTC)*. IEEE, 1–8.
- [18] Wang Wenping et al. 2009. The Impact of NBTI Effect on Combinational Circuit: Modeling, Simulation, and Analysis. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 18, 2 (2009), 173–183.
- [19] Zhang Fan et al. 2022. DARPT: Defense Against Remote Physical Attack Based on TDC in Multi-Tenant Scenario. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 559–564.
- [20] Zussa Loic et al. 2013. Power Supply Glitch Induced Faults on FPGA: An In-Depth Analysis of the Injection Mechanism. In *2013 IEEE 19th International On-Line Testing Symposium (IOLTS)*. IEEE, 110–115.
- [21] Zussa Loic et al. 2014. Efficiency of a Glitch Detector Against Electromagnetic Fault Injection. In *2014 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.
- [22] Zhang Maoshen et al. 2021. A Digital and Lightweight Delay-Based Detector Against Fault Injection Attacks. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.

Sharry: An Efficient and Sharing Far Memory System

Chen Chen
Zhejiang University
Hangzhou, China
chenc68@zju.edu.cn

Yuhang Huang
Zhejiang University
Hangzhou, China
22351093@zju.edu.cn

Shuiguang Deng
Zhejiang University
Hangzhou, China
dengsg@zju.edu.cn

Jianwei Yin
Zhejiang University
Hangzhou, China
zjuyjw@cs.zju.edu.cn

Xinkui Zhao*
Zhejiang University
Hangzhou, China
zhaoxinkui@zju.edu.cn

Abstract

Far Memory System(FMS) allows applications to access memory on remote machines(called memory nodes). However, existing FMSs can't deal with large loads and have low efficiency in utilizing far memory, which leads to the inability to share memory nodes among multiple processes, limiting the scalability of FMS. In this paper, we propose Sharry, an efficient Sharing FMS. Sharry manages memory objects from multiple processes within a unified address space, avoiding the overhead of space switching. Sharry also optimizes the utilization of far memory with fine-grained memory management. Additionally, Sharry offloads memory allocation to dedicated CPU core in order to handle larger loads in the sharing scenario. Compared to state-of-the-art FMS, Sharry improves memory utilisation by 45%, causing only 9% performance degradation when multiple processes sharing single memory node.

ACM Reference Format:

Chen Chen, Yuhang Huang, Shuiguang Deng, Jianwei Yin, and Xinkui Zhao. 2024. Sharry: An Efficient and Sharing Far Memory System. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3656508>

1 Introduction

Memory is the most critical hardware resource in today's data centres. However, data centres face two major challenges in providing memory. Firstly, the memory capacity of a single machine is insufficient to meet the growing demands of modern data centre applications, such as Deep Learning and Data Analysis. The data sets are often too large to fit into the RAM, which is known as the **Memory Capacity Wall**. Secondly, commodity servers combine memory capacity and the number of CPU cores in a fixed ratio, which leads to **Memory Stranding**[7]: memory fragments on servers cannot be utilized either due to the CPU cores being fully occupied or the memory fragment being smaller than requirements. A report from Google's data centre[14] reveals that the average memory utilization per server is only 60%, with a significant amount of memory

sitting idle. The root cause lies in the inability of applications to utilize memory across multiple machines.

To address this dilemma, One promising solution is **Memory Disaggregation**[1] in data centres. CPU cores in disaggregated data center can be paired with either local or remote RAM modules, and a high-speed network interconnect ensures low access latency for far memory. Currently, hardware vendors are developing new hardware architectures and interconnects to support memory disaggregation. However, rolling out new hardware in data centres can be costly and time-consuming. In contrast, far memory (FM) solution is built on a set of existing commodity server machines, while computing nodes(CN) running computing processes and memory nodes(MN) providing far memory.

Far memory can be accessed by applications through various approach. In **Page-Based approach** (such as FastSwap[2]), the OS kernel treats far memory as a swapping device, swapping between local and far memory at the granularity of 4KB memory pages. However, page-based approach suffers from read-write amplification, as well as kernel overheads (such as page-fault and address space switching). Furthermore, developers are unable to control performance, cause page-based FMS is transparent to applications. **Object-based approach** (like AIFM[13]) provide developers with remoteable pointer abstraction through user-level runtime, which triggers swapping at object granularity when the remoteable pointer is dereferenced or when high memory pressure is detected. The object-based approach effectively mitigates the read-write amplification and avoids kernel overheads. Moreover, it offers developers more flexibility in optimizing their applications.

Existing FMSs follow the assumption that far memory has sufficient capacity. Therefore, they utilize a coarse-grained far memory management pattern(as shown in Figure 1), where memory on local machine serves as a cache for far memory. When applications construct a remoteable object, FMS runtime generates copies of the data in both local memory and far memory. These copies won't be recycled in far memory until the remoteable object is deconstructed, we call it **Multi-Copy Pattern**. However, Multi-Copy Pattern introduces significant data redundancy and significant synchronization overhead between the multiple copies, which not only leads to lower utilisation of far memory, but also imposes limitations on the scale of remoteable objects in FMS.

Multi-Copy Pattern also presents a potential issue where during object swapping, far memory managements(like allocation and free) are not frequently executed as the objects' data copies were already allocated in far memory. Therefore, existing FMSs adopt naive approaches for managing far memory. they utilizes locks to ensure metadata consistency between threads during concurrent

*Corresponding author: zhaoxinkui@zju.edu.cn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06...\$15.00

<https://doi.org/10.1145/3649329.3656508>

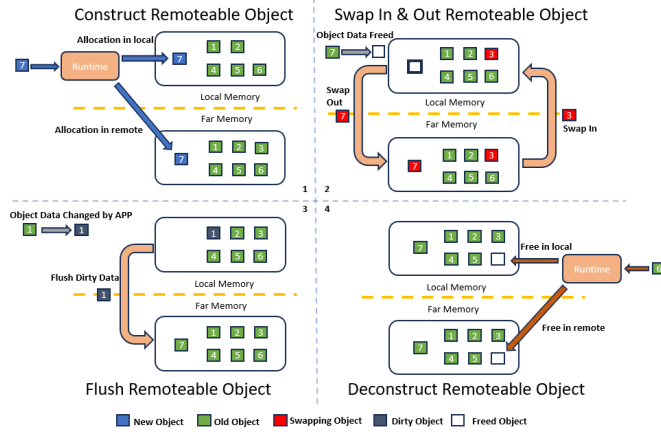


Figure 1: the lifecycle of remoteable objects in Multi-Copy Pattern

memory management. However, the overhead of inter-thread synchronization severely limits the efficiency of managing far memory, particularly in managing large-scale objects.

We propose **Sharry**, a far memory approach that enables efficient sharing of far memory among multiple processes. Similar to AIFM, Sharry provides users with remoteable pointers at the application level to access far memory. In contrast to multi-process operating systems that allocate a virtual memory address space for each process, Sharry implements a unified far memory address space, which allows objects from different processes to be managed in single address space, avoiding the overhead of address space switching when serving different processes. Unified far memory address space also makes it possible for interconnection between processes through far memory.

In contrast to Multi-Copy Pattern, Sharry leverages **Unique-Copy Pattern** to improve the efficiency of utilizing far memory. In Unique-Copy Pattern, each object in the system only has a Unique data copy. When an object is swapped in to local memory, its data copy in far memory is promptly freed. By adopting Unique-Copy Pattern, Sharry effectively resolves the data redundancy issue inherent in Multi-Copy Pattern and avoids the synchronization between multiple copies, which significantly improves the efficiency of utilizing far memory. However, Unique-Copy Pattern introduces a new challenge. In Unique-Copy Pattern, FMSs have to manage memory for each object swap, with a much higher frequency compared to Multi-Copy Pattern. Existing design of far memory allocator becomes a performance bottleneck of FMS.

To address this problem, we have implemented the **Sharry Allocator**. Firstly, Sharry Allocator adopts the **Separated Memory Layout**[4], which allows for faster identification of free memory blocks by decoupling the metadata from the data, ensures performance of Sharry when involving large-scale objects. Secondly, Sharry Allocator offloads memory management to an allocator thread that runs on a dedicated CPU core. Other threads have to request the allocator thread to do allocation or free. Sharry Allocator minimizes inter-thread synchronization overhead by eliminating redundant inter-thread synchronization, except the atomic primitives involved in interactions with the allocator thread.

Single CPU core struggles to handle the pressure of managing the entire unified memory address space, especially when far memory has a significantly large capacity. Sharry Allocator divides the unified address space into multiple Memory Sections of equal length.

Each Memory Section is assigned a dedicated CPU core to run its respective section allocator thread. We have also implemented a simple request scheduler to balance the memory pressure between different memory sections. This approach not only alleviates the computational burden on a single CPU core but also maximizes the utilization of computational resources within MN.

We implemented Sharry on regular servers. Compared to AIFM[13] and FastSwap[2], Sharry saves on average 45% of far memory when multiple processes share a single memory node (whose request pressure and object scale multiple times greater than single process.), causing at most 9% performance degradation.

The major contributions of Sharry are as follows:

- We propose a unified far memory address space design for managing objects from multi-process and avoiding the overhead of address space switching;
- We propose Unique-Copy Pattern, which effectively avoid data redundancy and synchronisation overheads between multiple copies, improve the efficiency of far memory usage;
- We propose Sharry Allocator, which effectively handle the significant memory management pressure brought by Unique-Copy Pattern through offloading memory management to multiple CPU cores, ensuring the performance of Sharry when multiple processes share single memory node;
- We used multiple benchmarks to comprehensively evaluate the performance of Sharry.

2 Background

Far Memory Far memory is a promising solution in data centres' architecture. One common approach to building far memory systems is page-based memory swapping. InfiniSwap[5] is the first RDMA-based far memory swapping system, while FastSwap[2] improves its performance through better scheduling and polling mechanisms. Leap[9] takes a different approach by prefetching memory pages based on the majority access pattern of processes, thus avoiding far memory accesses on critical paths. The page-based scheme is application-agnostic and transparent to the application. In addition to the page-based approach, another far memory approach swaps at the granularity of objects and exposes far memory to developers as remoteable pointers. AIFM[13] introduces a new programming model for far memory, including features such as remoteable pointers and dereference scopes. Carbink[16] introduces an efficient fault-tolerance mechanism in the far memory system and swaps at a spanning granularity instead of the object granularity strategy used in AIFM. Carbink also performs reclamation of far memory. Mira[6] is the most recent work that utilizes program analysis to improve prefetching on top of AIFM.

Existing work often overlooks the scalability of far memory systems and fails to optimize the efficiency of far memory management. our work focuses on addressing these issues and aims to improve the scalability and efficiency of far memory management.

Memory Management Research on local memory management has been well-established. Today's application relies on user-level memory allocators, such as PTmalloc2[3](From glibc), TCMalloc[4](From Google), and Mimalloc[10](From Microsoft), for efficient memory management. Domain-specific implementations, such as NValloc[15], are also available for non-volatile memory. These memory allocators typically support concurrent memory management for multi-threaded programs, efficiently handling simultaneous memory requests from different CPU cores. However, as the number of CPU cores continues to increase, software memory

allocators face challenges in managing **Thread Contention**. The synchronization of inter-CPU core metadata through atomic operations or locks, which maintains the global free list, introduces significant performance overheads. To address this problem, TCMalloc utilizes per-CPU/thread caches to manage metadata associated with each logical core, reducing the need for locks during memory allocations and deallocations. Mimalloc employs a three-page locally-shared list of frees to enhance locality, minimize contention, and support fine-tuning on fast-path allocations and frees.

Inspired by NextGenMalloc[8], Sharry addresses the Thread Contention problem in large-scale request scenarios by offloading memory management to a dedicated CPU core. This approach allows for serial execution of memory allocation requests, eliminating the synchronization overhead associated with atomic primitives.

3 Motivation

Traditional memory management approach, such as the buddy allocator and slab allocator in the kernel, as well as user-level allocators, need to be reevaluated in far memory. In these scenarios, allocation requests come from different computing processes on different servers, requiring scalability to handle large memory capacities and process requests while managing fragmentation. There are two potential solutions, first one is **Centralized Solution**, which centralizing all allocation requests on a huge memory node, second one is **Distributed Solution**, which distributing memory allocation across multiple nodes, requiring efficient coordination.

Our KEY INSIGHT is that **Centralized Solution using a high-capacity commodity machine as partial memory pool (e.g., at the rack level) is more suitable for today's data centres instead of Distributed Solution that clusters plenty small machines to construct a global memory pool**. This is because the Distributed Solution faces three major challenges:

- provisioning memory resources in a distributed cluster is a complex process, even for most advanced cluster scheduler;
- running far memory runtime on every memory node is resource-intensive, while a centralized scheme only requires running one far memory runtime;
- having multiple memory nodes corresponding to a computing process increases the complexity for the computing side runtime as it needs to determine which memory node to swap out;

hence Centralized Solution is simpler and more efficient, easier to rolling-out in data centres (requiring few machines).

Sharry aims to effectively share far memory(provided by single memory node) among multiple computing processes and optimizes the performance of FMS in centralized and large-scale scenarios. We will introduce the design of Sharry in next section.

4 System Design

Figure 2 illustrates the overall architecture of Sharry, which comprises multiple computing nodes and a single memory node. Each computing node executes computing processes that may utilize far memory upon computing side runtime of Sharry. Memory node executes the memory side runtime of Sharry, exposing far memory through network communication. Notably, Sharry does not require any custom hardware or modifications to OS kernel, is able to run on any commodity servers. Computing processes swap out objects when runtime detects memory pressure, and swap objects back into the local memory when accessed.

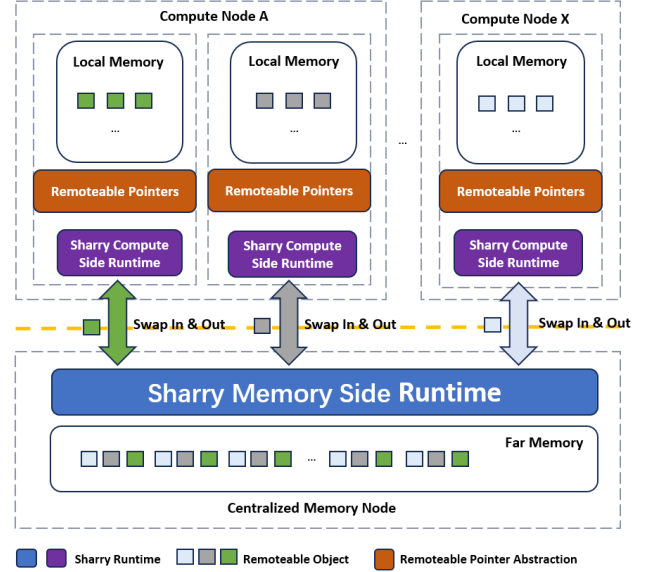


Figure 2: Sharry's Overall architecture.

Inspired by previous work AIFM[13], Sharry utilizes remoteable pointers abstraction to expose far memory at the object granularity, which also enables constructing remoteable data structures to execute more complex workloads.

When serving multiple computing processes, Sharry maintains a **Unified Memory Address Space(UMA)** instead of creating separate address spaces and manage all objects in UMA. Unlike the previous works which follow the Multi-Copy Pattern, Sharry employs the **Unique-Copy Pattern** to increase the efficiency of utilizing far memory. Within UMA, Sharry manages far memory through **Sharry Allocator**, which offloads memory management to a single dedicated CPU core and running as allocator thread. Sharry Allocator avoids multi-threads contention caused by concurrent memory management.

To further enhance the performance of Sharry Allocator running on a dedicated CPU core, Sharry divides UMA into multiple memory sections. Each memory section corresponds to an allocator thread running on a dedicated core, improving the efficiency of memory management through the divide-and-conquer strategy, without causing thread contention.

4.1 Unified Memory Address Space

Sharry's memory side runtime is tasked with managing all the memory within the memory node and making it accessible to computing processes through the network. When multiple computing processes share a single memory node, Sharry introduces an efficient design called **Unified Memory Address Space**(as shown in figure 3), which aims to consolidate the memory management of remoteable objects from all computing processes.

Address Space Design Different from the design of the process address space in a multi-process operating system that each process has its own virtual memory address space and page table, ensuring isolation between different processes. In our design, Sharry has only one memory address space(UMA) and Sharry allocates addresses within UMA for objects from all processes, instead of creating a separate "remote address space" for each process, which would introduce the overhead of switching address spaces when serving different computing processes.

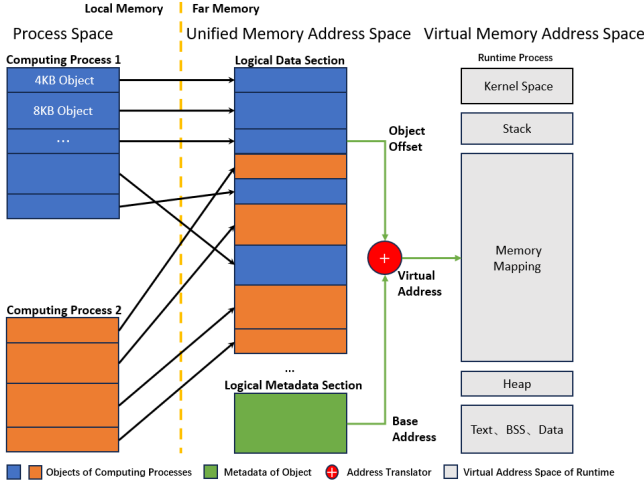


Figure 3: Unified Memory Address Space

As depicted in Figure 3, the unified memory address space is a logical address space provided by the Sharry runtime. It has a size equal to the physical memory size of the memory node (subtracting the size of the kernel space), in contrast to traditional memory address space, UMA only consists of metadata section and object data section, without separate divisions for heap, stack and others. The object data section stores data, and metadata section stores all objects' metadata. We will introduce this separate layout for metadata and data in Section 4.3.

Flat Address Translation Sharry's memory side runtime is also responsible for translating addresses from the *Unified Memory Address Space* (UMA) to the *Virtual Memory Address Space* (VMA) of the runtime process. Then, MMU (Memory Management Unit) will translate the Virtual Memory Address to the physical address of the memory node. Since the size of UMA is similar to the size of VMA, Sharry chooses to build a direct mapping from UMA to VMA at the byte granularity (as shown in figure 3), instead of emulating traditional memory systems that use multi-level page table mappings at the page granularity. This flattened mapping approach limits address translation to a maximum of one address lookup.

4.2 Unique Copy

Sharry leverages the Unique-Copy Pattern to utilize far memory efficiently. In this Pattern, an Object has only one globally unique copy present in both local and far memory. When an Object is swapped out to far memory, the memory side runtime promptly allocates a memory block of the appropriate size on the memory node, leading to reclamation of the Object in local memory. On the other hand, when an Object is swapped into local memory, the memory side runtime reclaims the memory block in far memory. This mechanism enhances the efficiency of far memory utilization.

To avoid inefficiencies in object swapping caused by frequent memory block reclamations, Sharry employs an asynchronous reclamation approach. Memory block reclamation is performed only when the total number of memory blocks need to be freed reaches a threshold. By adopting the Unique-Copy Pattern, Sharry eliminates the synchronization overhead between multiple copies of data, which caused by Multi-Copy Pattern.

4.3 Sharry Allocator

Existing far memory systems, such as AIFM[13], typically adopt the Multi-Copy Pattern for far memory management. In this Pattern,

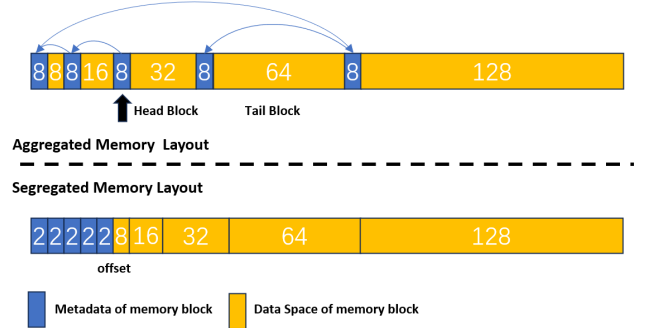


Figure 4: Segregated Layout and Aggregated Layout

memory allocation is performed once when a remoteable object is constructed, and deallocation is performed once when the object is reclaimed. However, Sharry introduces the Unique-Copy Pattern, which involves intense memory management throughout the life-cycle of a remoteable object. This distinction highlights that the primitive memory allocators utilized in existing FMS have become a performance bottleneck.

Existing memory allocators, such as PTmalloc2[3], TCMalloc[4], and Mimalloc[10], are designed for single process scenarios with limited memory block sizes and memory request intensities. Two key issues affecting memory allocator's performance are **memory block scales** and **thread contention**. The performance degrades as the scales of the memory block increases. Thread contention arises from the concurrent memory management support in existing memory allocators, where multiple threads running on different CPU cores concurrently process requests. Synchronization between threads necessitates mutex locks to maintain a global free list, resulting in significant performance degradation. As the number of CPU cores continues to grow, memory managers face even greater challenges in handling thread contention. We introduce Sharry Allocator, which is designed to optimize the performance bottleneck of memory allocators, which is amplified in far memory scenarios.

Decoupled Memory Layout The Unified Memory Address Space design in Sharry provides a straightforward and comprehensible approach for managing the far memory address space. However, as the scale of memory management requests from computing processes increases, Sharry's efficiency in handling large memory blocks can potentially become a runtime performance bottleneck due to the growing number of memory blocks involved. To address this issue, Sharry adopts a metadata and data decoupled Segregated Memory Layout. This design enhancement aims to improve Sharry Allocator's performance when facing large-scale Objects.

As illustrated in Figure 4, previous studies on memory space have proposed two prominent metadata layouts: the **Aggregated Layout** (utilized by Mimalloc[10]) and the **Segregated Layout** (used by TCMalloc[4]). In Aggregated Layout, the first 8 bytes of each free block (assuming a 64-bit size) serve as a pointer to the next free block. In contrast to Aggregated Layout, the Segregated Layout decouples the metadata from the memory block's data, using a smaller index (e.g., 16 bits) to indicate the location of the next free block. This design choice enhances memory utilization efficiency and improves memory management performance by solely traversing the metadata portion of the memory space, without the need to navigate through the entire memory space. While there is always a

trade-off between the Aggregated Layout and the Segregated Layout, the Segregated Layout is better suited for managing large-scale memory blocks in far memory scenarios.

Offload to Dedicated Core In the current design of memory allocators, contention can arise when two threads simultaneously manage memory metadata. To mitigate this contention and control the order of changes made to critical resources, read-write locks are commonly used. Sharry Allocator offloads memory management to a dedicated CPU core, uses a single-threaded mode to avoid synchronizing between threads.

In this design, Only allocator thread running on dedicated CPU core have the ability to operate on the metadata of memory blocks. This approach effectively eliminates most thread contention, minimize the overheads caused by locks. Ordinary threads perform memory management by modifying the synchronization variables using atomic primitives and requesting the allocator thread to handle the request on their behalf.

The performance advantages of Sharry Allocator stem from two key aspects. Firstly, since all memory management operations runs on a dedicated core, it ensures sequential execution of all request. As a result, there is no need for any locks in Sharry Allocator to synchronize metadata between threads, thus reducing the overhead. Secondly, the dedicated core exclusively runs allocator threads and does not perform thread switching, thereby preventing CPU cache line contamination.

4.4 Multi-Section Memory Management

Sharry employs a design where all memory management operations are offloaded to a single thread running on a dedicated core. This approach reduces thread contention during memory management and ensures sequential execution of all memory requests on that specific core. However, in large-scale scenarios, a single core may not possess sufficient computational capability to handle all requests, resulting in performance degradation. To overcome this limitation, Sharry introduces the Multi-Section design, which dividing the unified memory address space into multiple memory sections. For each memory section, Sharry allocates a dedicated core to run a allocator thread. Each thread is responsible for managing memory within the corresponding memory section(as shown in figure 5). This design allows for better utilization of computing resources on the memory node and alleviates computing pressure on allocator threads running on each dedicated core.

In the case of memory management requests from multiple processes, Sharry employs a request scheduler to dynamically select the most suitable memory section. The request scheduler performs request scheduling, and the corresponding allocator thread of the selected memory section handles the incoming requests. Request scheduler is able to monitor the usage pressure of each memory section and the request pressure during a fixed time window in real-time. This enables the scheduler to balance the memory usage and computing pressure across different memory sections.

5 Implementation

We implemented the Sharry runtime library in C++, comprising approximately 7.7K lines of code (LOC), which runs on both local nodes and memory nodes. Sharry is designed to run on unmodified generic operating systems. For the memory nodes, we implemented Sharry Allocator which offloads memory management to a dedicated core and handles the synchronization of request threads and allocator threads by atomic primitives. Sharry Allocator are used

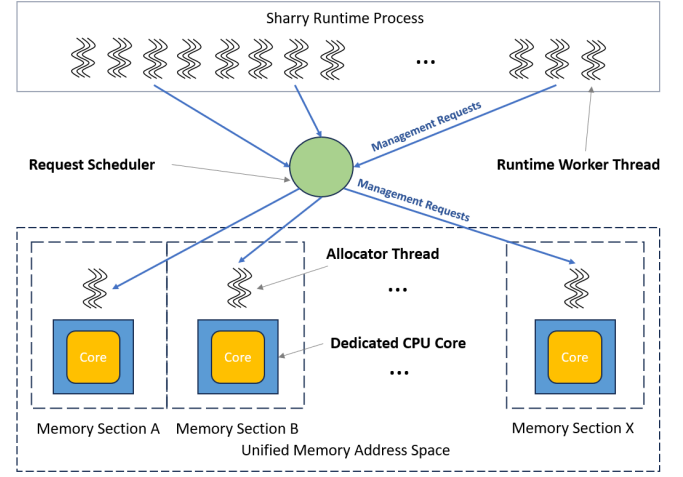


Figure 5: Multi-Section Memory Management

for memory management within unified memory address space. The communication between computing nodes and memory nodes occurs over TCP network stack. Currently, Sharry supports multiple computing nodes and a single memory node.

6 Evaluation

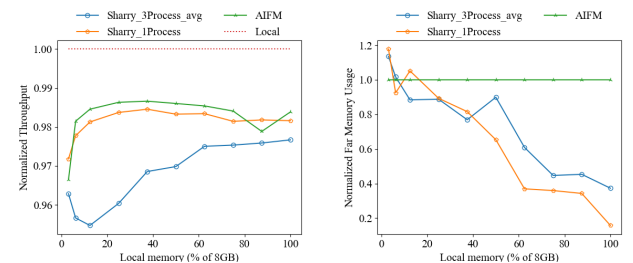
The experiments are designed to answer following questions:

- What is the throughput and far memory usage of Sharry when multiple processes sharing far memory(section 6.1)?
- How does sharry perform when there are massive objects in the system(section 6.2)?
- How does Sharry's performance compare to the to standard Linux and AIFM(section 6.3)?

Testbed setup Our experimental environment consists of four commodity servers, with three nodes serving as computing nodes and one node serving as a memory node. All nodes are equipped with 10-core Intel Xeon E5-2640 v4 CPUs (2.40 GHz), 64GB of memory, and connected by 25 GbE network. We are using Ubuntu 20.04 (kernel v5.14). We followed common practices to tune these servers for low latency[12], including disabling CPU frequency scaling, machine-check exceptions, and transparent hugepages.

6.1 Sharing vs. Not Sharing

We use a micro-benchmark to test the performance of Sharry when multiple computing processes share far memory. The benchmark is based on an array data structure, where we create 8 remoteable arrays. For each we set the input size to 1GB, including 260,000 objects of 4KB size. We run this benchmark on a cluster consists of three computing nodes and one memory node.



(a) Throughput in sharing (b) memory usage in sharing
Figure 6: 3 processes sharing single memory node

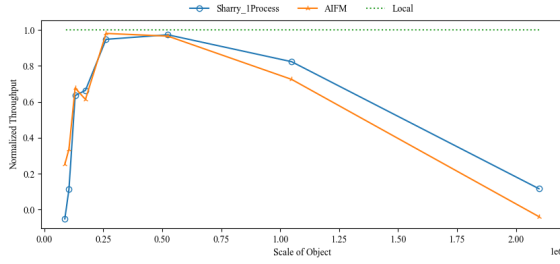


Figure 7: Performance in different Scale.

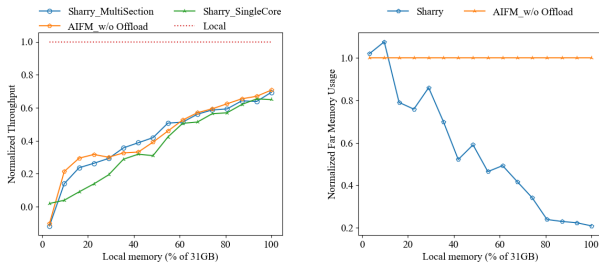
Figure 6 shows the performance of Sharry when sharing far memory and the efficiency of utilising far memory. As shown in Figure 6b, as the local memory size increases, the Unique-Copy Pattern helps Sharry save more far memory compared to AIFM, up to 50%. The delayed reclamation mechanism of Sharry Allocator causes Sharry to use slightly more far memory than the total workload size minus the local memory size. Figure 6a shows that Sharry’s performance causes up to 9% performance degradation when sharing far memory, and the performance degradation gradually decreases as the local memory size increases.

6.2 Impact of Scale

We modify the benchmark above slightly by fixing the local memory size to 2GB and changing the object scale in the system. Figure 7 shows the performance of Sharry as the scale of the objects stored in FMS continues to grow. When the scale is large, Sharry has a visible performance improvement compared to AIFM (nearly 10%), which we believe comes from the Segregated Layout.

6.3 Real Application Benchmark

We also tested sharry’s performance on a real application benchmark, DataFrame[11], a data analysis system written in 24.3K LOC C++, which is widely used as the current infrastructure in the field of data analysis. DataFrame can be both computationally intensive and memory intensive when operating on large datasets, making it well suited to use far memory. We build our benchmark on 2016’s NYC Taxi Trip dataset, using DataFrame to analyses and predicts taxi travel times at different periods of day.



(a) Throughput in DataFrame (b) Memory usage in DataFrame
Figure 8: DataFrame Performance.

Figure 8b shows the usage of shared far memory. Figure 8a shows the throughput of Sharry and AIFM (without function offloading) on DataFrame. Unique-Copy Pattern does impact on Sharry’s performance. When the local memory is small, due to intense swapping requests, Sharry’s throughput is slightly lower than AIFM (about 9% performance degradation). The design of **Multiple Sections** brings significant performance improvement compared to **Single Section**, as it distributes the computing load. As the size of local memory increases, Sharry’s performance getting close to AIFM. Even local memory reaches 100%, both Sharry and AIFM still lag behind Local cause the high dereferencing cost of remoteable pointers.

Discussion: How to offload function in FMS? Existing FMS are discussing how to improve performance by offloading function to memory node, cause instead of swapping large amounts of data into local memory, processing data directly in far memory avoids network overhead and makes efficient use of the CPU cores on the memory node (AIFM achieves near-local memory performance when performing offloading functions of Dataframe).

For Unique-Copy Pattern, the key problem of function offloading is that objects need to be calculated are not all in the local or remote. To address this, we have two approach, Firstly, **Gathering all objects in one place**. Secondly, **Calculating each part of data, and aggregating them**, we prefer the latter one, which avoids redundant network communication, but brings complexity. Our next work will discuss how to offload functions in FMS more efficiently.

7 Conclusion

In this paper, we propose Sharry, an object-based FMS. Sharry enables a single memory node to serve multiple computing processes and achieves efficient management of shared far memory through various techniques such as the **Unique-Copy Pattern**, **Separated Memory Layout**, and **Memory Management Offloading**. Compare to AIFM, Sharry achieves a 45% improvement in far memory utilization while incurring 9% performance loss at most.

8 Acknowledgment

We deeply appreciate the comments from the reviewers. This work was supported in part by the National Key R&D Program of China (2022YFF0902702), and the Major Program of National Natural Science Foundation of Zhejiang (LD24F020014), and the Zhejiang Pioneer (Jianbing) Project (2024C01032), and part of the Key R&D Program of Ningbo (2023Z235), and part of the Ningbo Yongjiang Talent Programme (2023A-198-G), and part of the Beijing Life Science Academy (BLSA:2023000CB0020).

References

- [1] Marcos K. Aguilera and Nadav Amit. 2018. Remote regions: a simple abstraction for remote memory. In *2018 USENIX Annual Technical Conference (ATC '18)*.
- [2] Emmanuel Amaro and Branner-Augmon. 2020. Can Far Memory Improve Job Throughput?. In *15th European Conference on Computer Systems (EuroSys '20)*.
- [3] Wolfram Gloger. 2022. PTmalloc2. <http://www.malloc.de/en/>.
- [4] Google. 2023. TCMalloc. <https://github.com/google/tcmalloc/>
- [5] Juncheng Gu and Youngmoon Lee. 2017. Efficient Memory Disaggregation with Infiniswap. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI '17)*.
- [6] Zhiyuan Guo, Zijian He, and Yiyang Zhang. 2023. Mira: A Program-Behavior-Guided Far Memory System (SOSP '23).
- [7] Huaicheng Li and Berger. 2023. Pond: CXL-Based Memory Pooling Systems for Cloud Platforms. In *28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2023)*.
- [8] Ruihao Li and Qinzhe Wu. 2023. NextGen-Malloc: Giving Memory Allocator Its Own Room in the House (HOTOS '23).
- [9] Hasan Al Maruf and Mosharaf Chowdhury. 2020. Effectively Prefetching Remote Memory with Leap. In *2020 USENIX Annual Technical Conference (ATC '20)*.
- [10] Microsoft. 2023. Mimalloc-bench. <https://github.com/daanx/mimalloc-bench/>
- [11] Hossein Moein. 2023. C++ DataFrame for statistical, Financial, and ML analysis. <https://github.com/hosseinmoein/DataFrame>
- [12] Amy Ousterhout and Joshua Fried. 2019. Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*.
- [13] Zhenyuan Ruan and Malte Schwarzkopf. 2020. AIFM: High-Performance, Application-Integrated Far Memory. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*.
- [14] Muhammad Tirmazi and Barker. 2020. Borg: The next Generation. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20)*.
- [15] Devesh Tiwari and Lee. 2010. MMT: Exploiting fine-grained parallelism in dynamic memory management. In *2010 IEEE International Symposium on Parallel and Distributed Processing (IPDPS '10)*.
- [16] Yang Zhou and Hassan. 2022. Carbink: Fault-Tolerant Far Memory. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*.